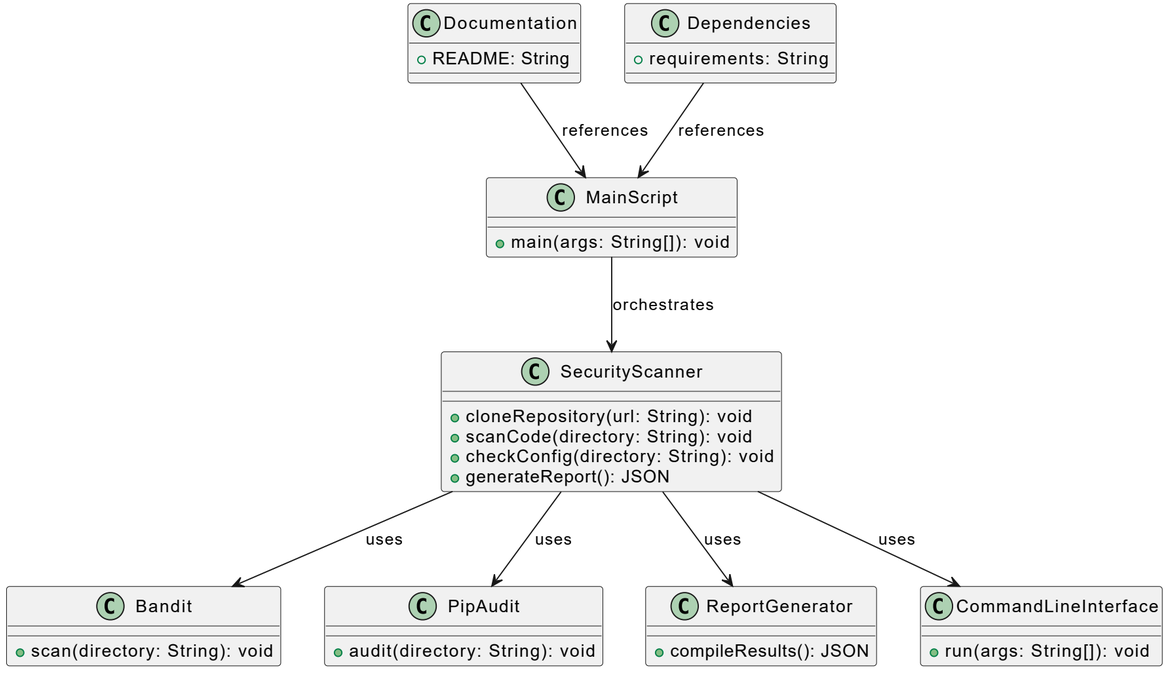
SASVA Project Scan Report

Git URL: https://github.com/guptaanurag/python-vulnerability-scaner | Branch: main

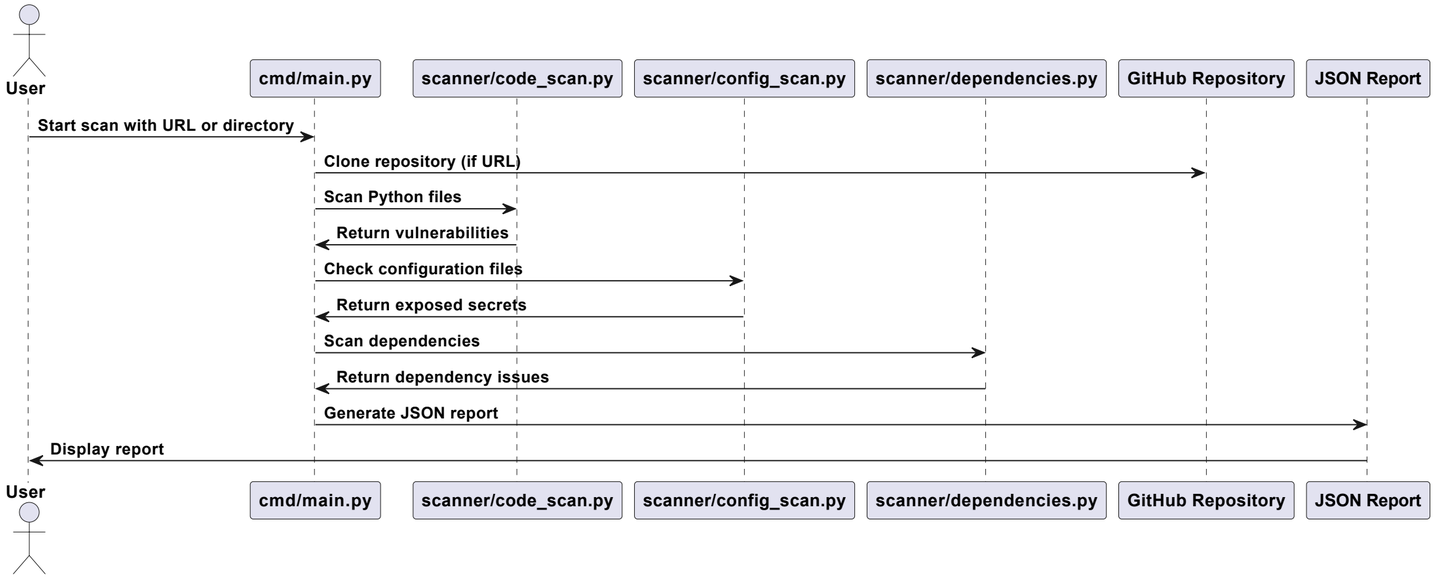
1) Class Diagram:

The class diagram represents the structure of the Python security scanner project. The main class, SecurityScanner, is responsible for cloning repositories, scanning code and configurations, and generating reports. It utilizes Bandit for code scanning and PipAudit for auditing dependencies. The ReportGenerator class compiles scan results into a JSON report. The CommandLineInterface class allows the tool to be run from the command line. The MainScript class serves as the entry point, orchestrating the scanning process. Documentation and Dependencies are auxiliary classes that provide project information and required packages, respectively.



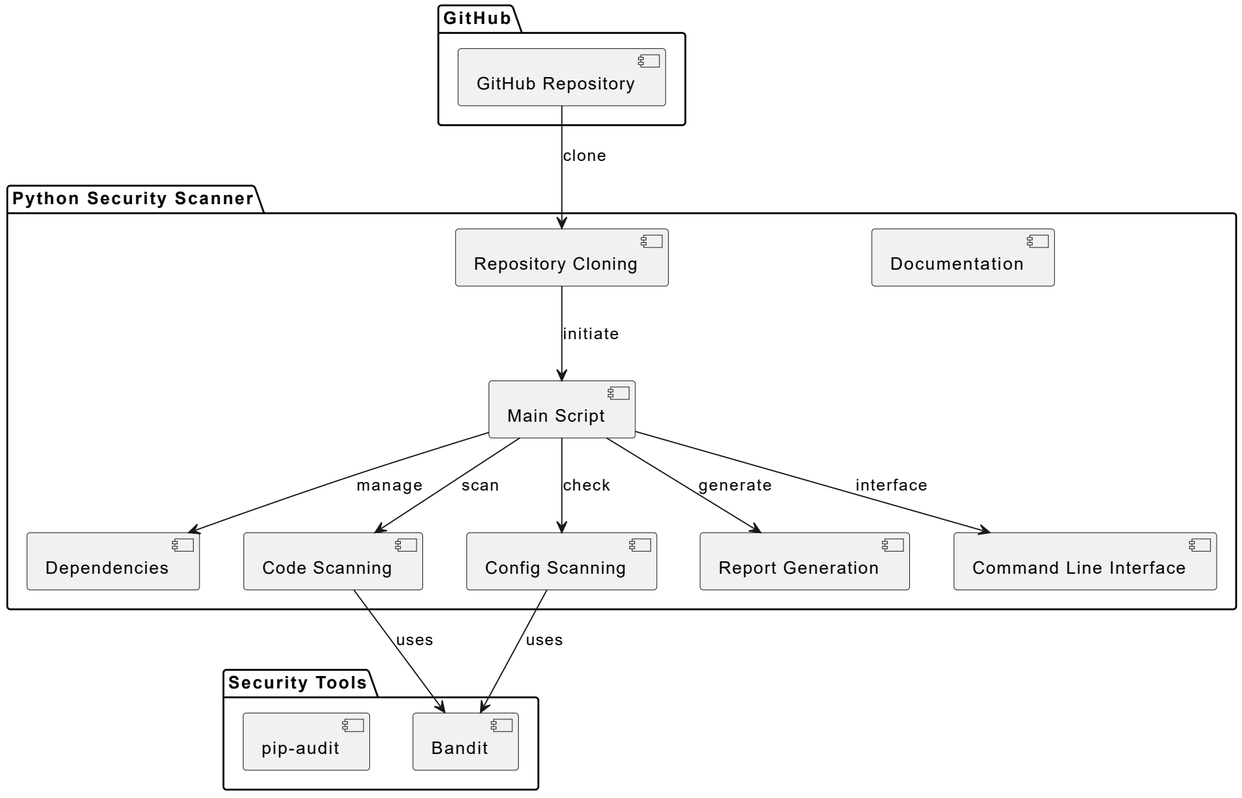
2) Sequence Diagram:

The sequence diagram illustrates the workflow of the Python security scanner. The user initiates a scan by providing a URL or local directory. The main script handles repository cloning if a URL is provided. It then calls the code scanning module to identify vulnerabilities in Python files, the configuration scanning module to detect exposed secrets, and the dependencies module to check for issues. Finally, the results are compiled into a JSON report, which is presented to the user.



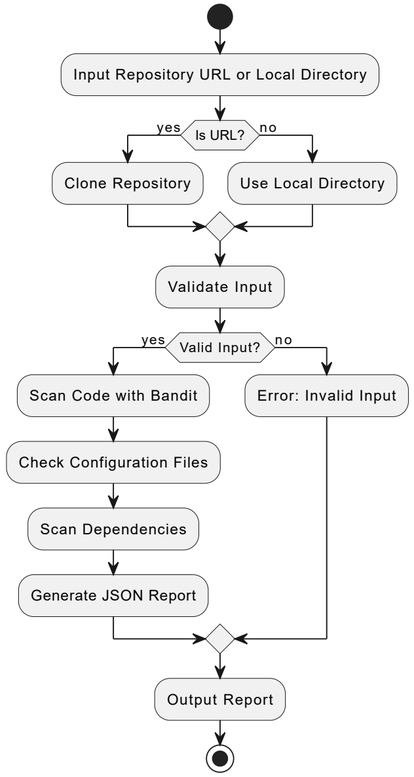
3) Logical Diagram:

The logical diagram represents the components and interactions within the Python Security Scanner project. The 'Main Script' component orchestrates the entire scanning process, which includes cloning repositories from GitHub, scanning code and configuration files, and generating reports. The 'Code Scanning' and 'Config Scanning' components utilize the 'Bandit' tool for security assessments. The 'Command Line Interface' allows user interaction, while 'Dependencies' are managed separately. The diagram ensures all components are correctly linked and named, adhering to PlantUML conventions.



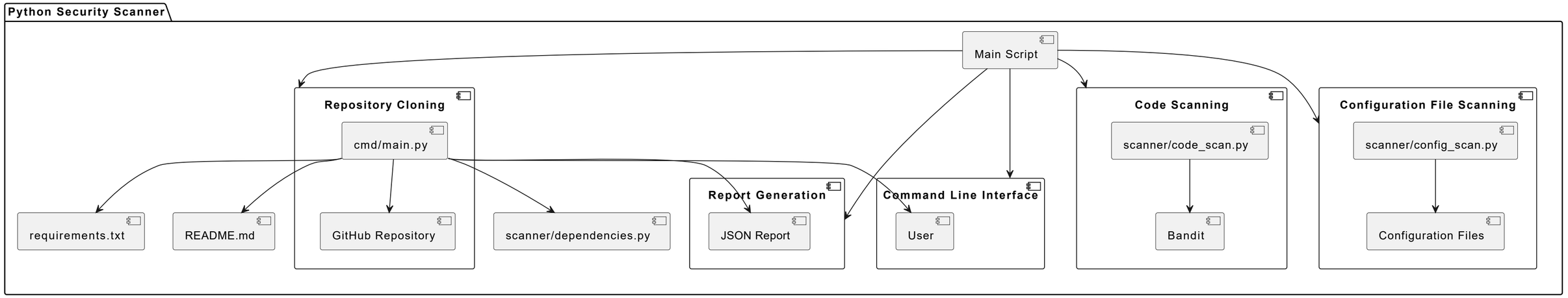
4) Activity Diagram:

The activity diagram illustrates the workflow of the Python security scanner. It starts by accepting a repository URL or a local directory as input. If the input is a URL, the repository is cloned; otherwise, the local directory is used. The input is validated, and if valid, the tool proceeds to scan the code using Bandit, check configuration files for exposed secrets, and scan dependencies. Finally, a JSON report is generated and outputted. If the input is invalid, an error is reported.



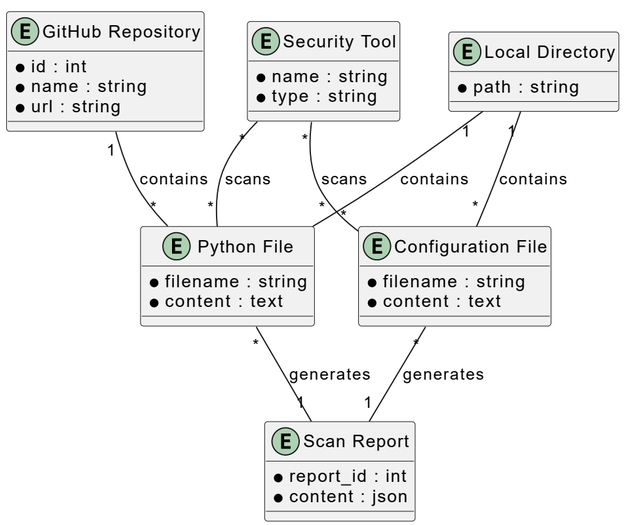
5) Component Diagram:

The component diagram illustrates the architecture of the Python Security Scanner project. The main script, 'cmd/main.py', orchestrates the entire scanning process, interacting with various components such as repository cloning, code scanning, configuration file scanning, and report generation. The 'scanner/code_scan.py' uses Bandit for scanning Python code, while 'scanner/config_scan.py' checks configuration files for exposed secrets. The results are compiled into a JSON report. The tool is designed to be run from the command line, facilitating integration into existing workflows. Dependencies and documentation are managed through 'requirements.txt' and 'README.md', respectively.



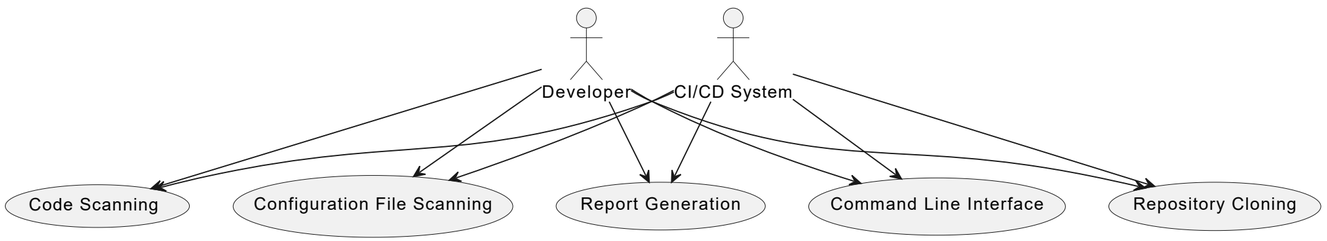
6) ER (Entity-Relationship) Diagram:

The ER diagram represents the relationships between different entities involved in the Python security scanner project. The 'GitHub Repository' and 'Local Directory' entities contain 'Python File' and 'Configuration File' entities, which are scanned by 'Security Tool' entities like Bandit and pip-audit. The results of these scans are compiled into 'Scan Report' entities. This diagram illustrates how the scanner processes files from repositories or local directories, uses security tools to perform scans, and generates reports based on the findings.



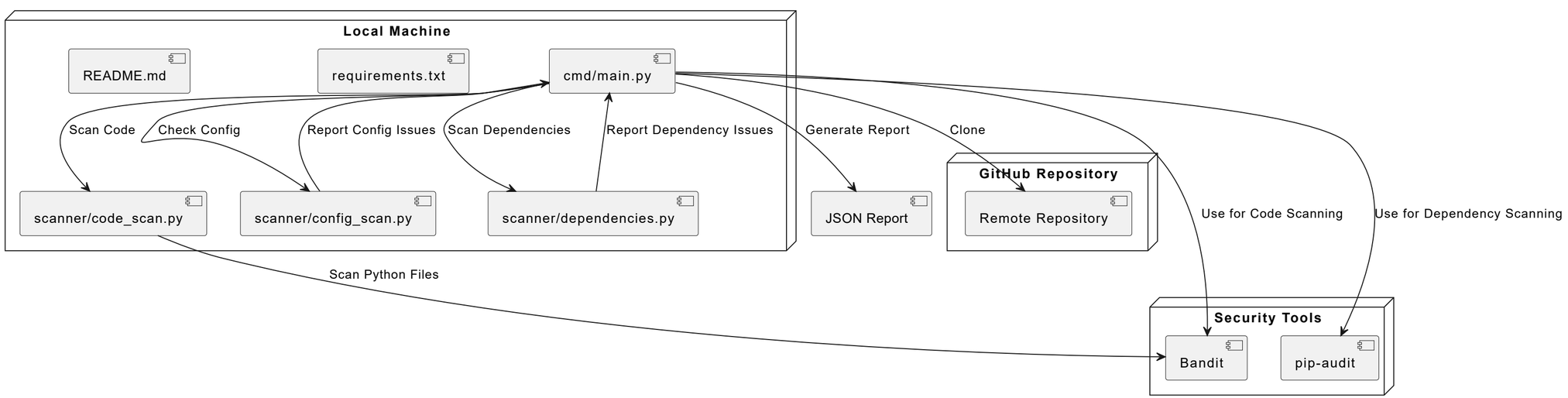
7) Use Case Diagram:

The use case diagram depicts the interactions between two actors, 'Developer' and 'CI/CD System', with the system's functionalities. Both actors can initiate the use cases: 'Repository Cloning', 'Code Scanning', 'Configuration File Scanning', 'Report Generation', and 'Command Line Interface'. These use cases represent the essential features of the system, enabling users to clone repositories, perform code and configuration file scans, generate reports, and utilize the command line interface for integration into workflows.



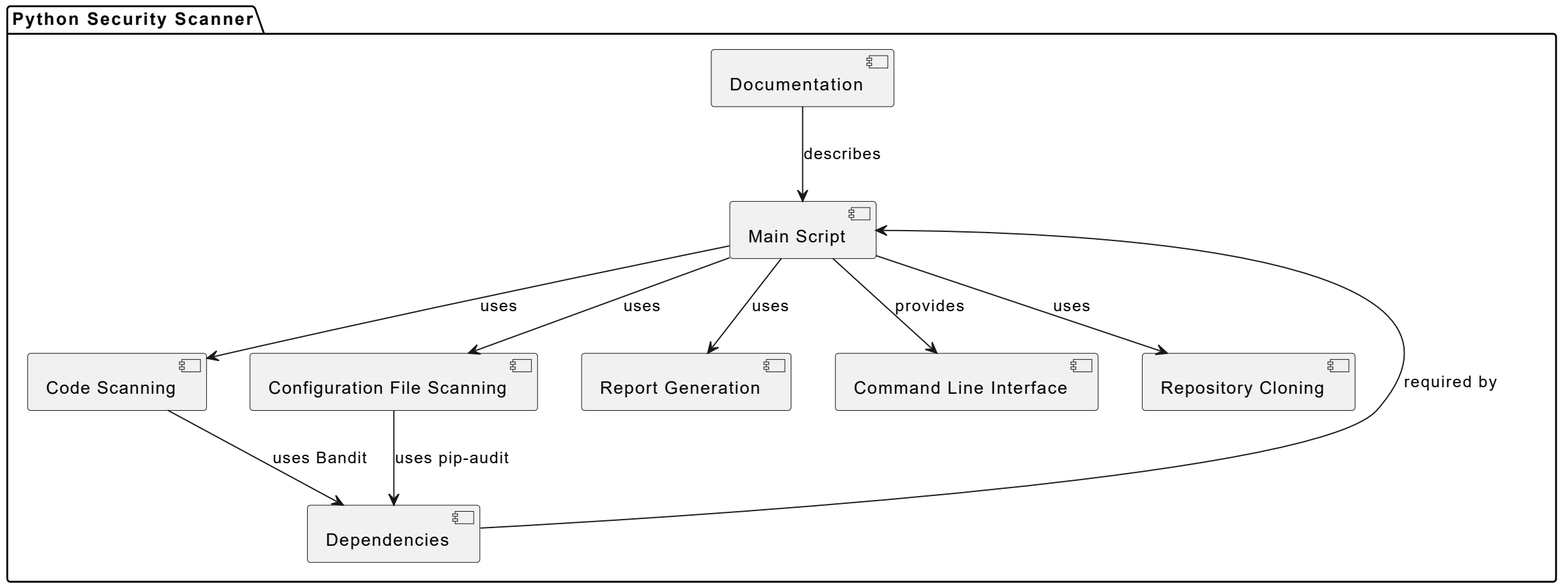
8) Deployment Diagram:

The deployment diagram illustrates the components involved in the Python security scanner project. The 'Local Machine' node contains the main script 'cmd/main.py', which orchestrates the scanning process by interacting with other scripts like 'scanner/code_scan.py', 'scanner/config_scan.py', and 'scanner/dependencies.py'. The 'GitHub Repository' node represents the remote repository that can be cloned for scanning. The 'Security Tools' node includes 'Bandit' and 'pip-audit', which are used for code and dependency scanning, respectively. The main script generates a JSON report after completing the scans.



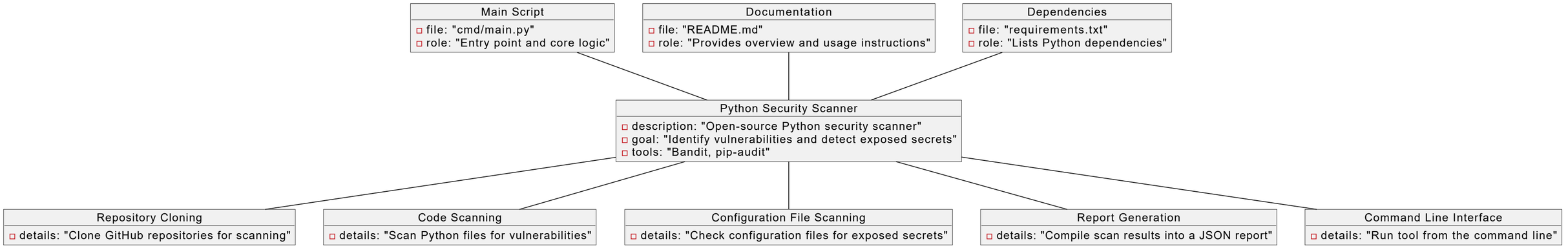
9) Composite Structure Diagram:

The composite structure diagram represents the Python Security Scanner project. The main script is the core component that interacts with other components such as code scanning, configuration file scanning, report generation, and repository cloning. The command line interface is provided by the main script for user interaction. Code scanning uses Bandit, and configuration file scanning uses pip-audit, both of which are dependencies listed in the project. Documentation describes the main script, and dependencies are required for the main script to function.



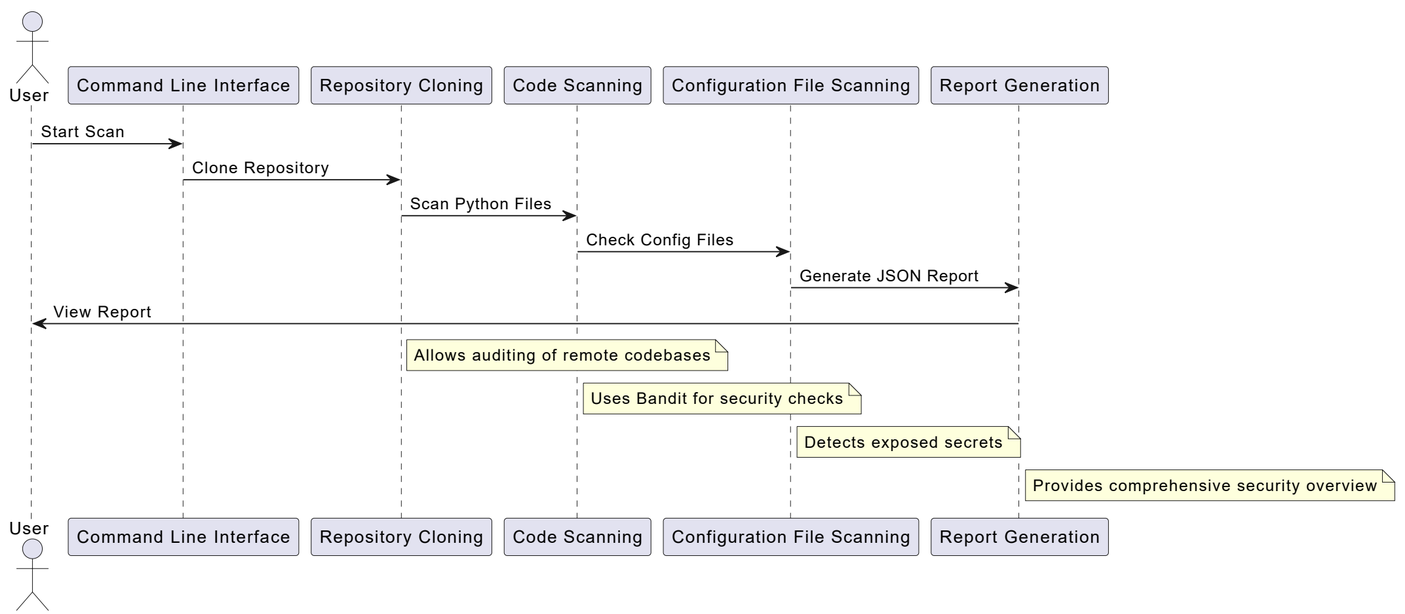
10) Object Diagram:

The object diagram represents the Python Security Scanner project, highlighting its main components and features. The central object is the 'Python Security Scanner', which interacts with various features such as 'Repository Cloning', 'Code Scanning', 'Configuration File Scanning', 'Report Generation', and 'Command Line Interface'. The 'Main Script' serves as the entry point, while 'Documentation' and 'Dependencies' provide necessary support for the project's functionality.



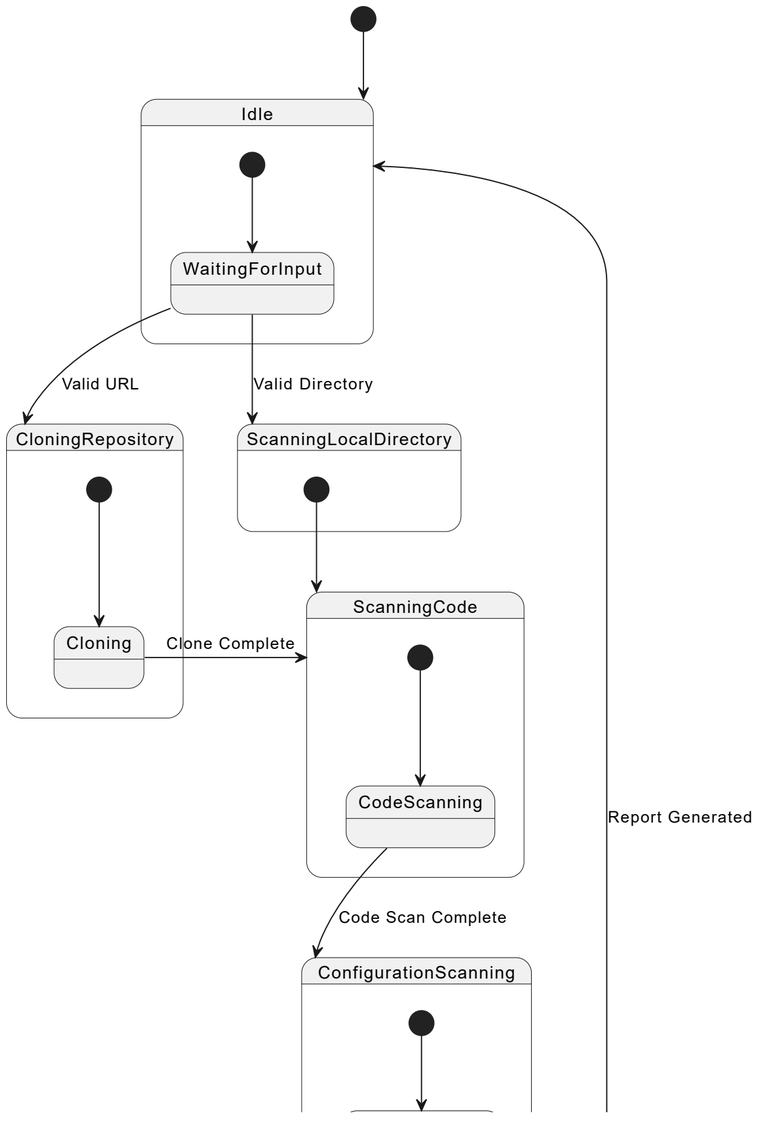
11) User Journey Map:

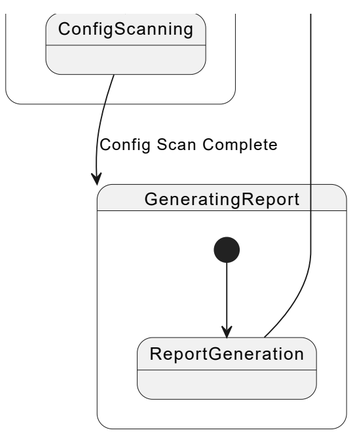
The user journey map illustrates the process a user follows to perform a security scan using the Python security scanner. The user initiates the scan via the command line interface, which triggers the repository cloning feature to download the codebase. The cloned repository is then scanned for vulnerabilities using Bandit, followed by a check for exposed secrets in configuration files. Finally, a JSON report is generated, providing the user with a detailed overview of the security status.



12) State Diagram:

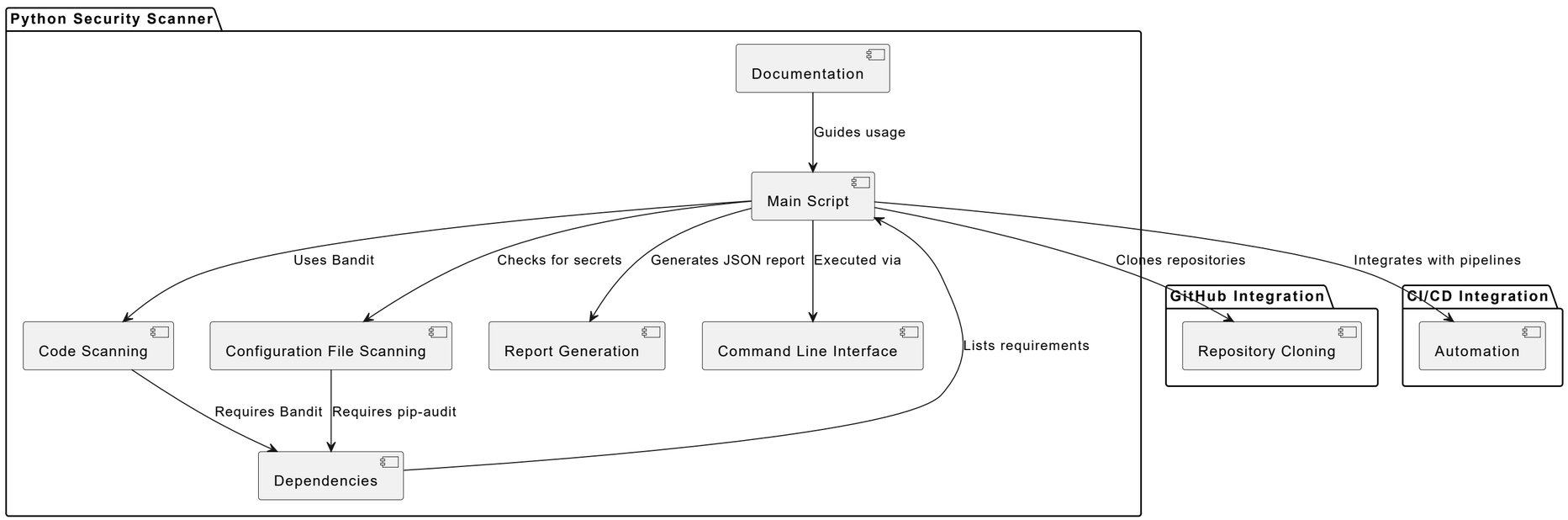
The state diagram represents the workflow of the Python security scanner. It starts in the Idle state, waiting for input. Upon receiving a valid URL, it transitions to CloningRepository, where it clones the repository and then moves to ScanningCode. If a valid local directory is provided, it directly transitions to ScanningCode. In ScanningCode, it performs CodeScanning followed by ConfigurationScanning. Once both scans are complete, it transitions to GeneratingReport, where a report is generated, and then returns to the Idle state.





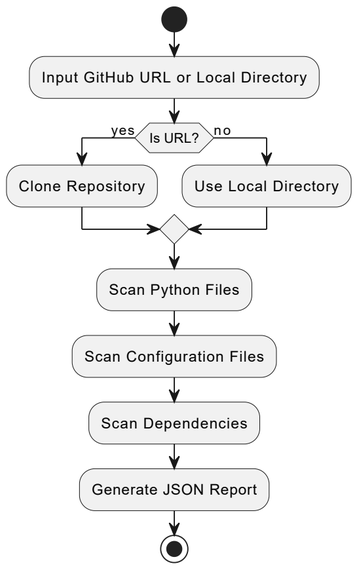
13) Architecture Diagram:

The architecture diagram illustrates the components of the Python Security Scanner project. The main script orchestrates the scanning process, utilizing Bandit for code scanning and pip-audit for configuration file scanning. It generates a JSON report summarizing the findings. The tool is executed via a command line interface and can clone GitHub repositories for scanning. Dependencies are managed through a requirements file, and the documentation provides usage guidance. The tool can be integrated into CI/CD pipelines for automated security checks.



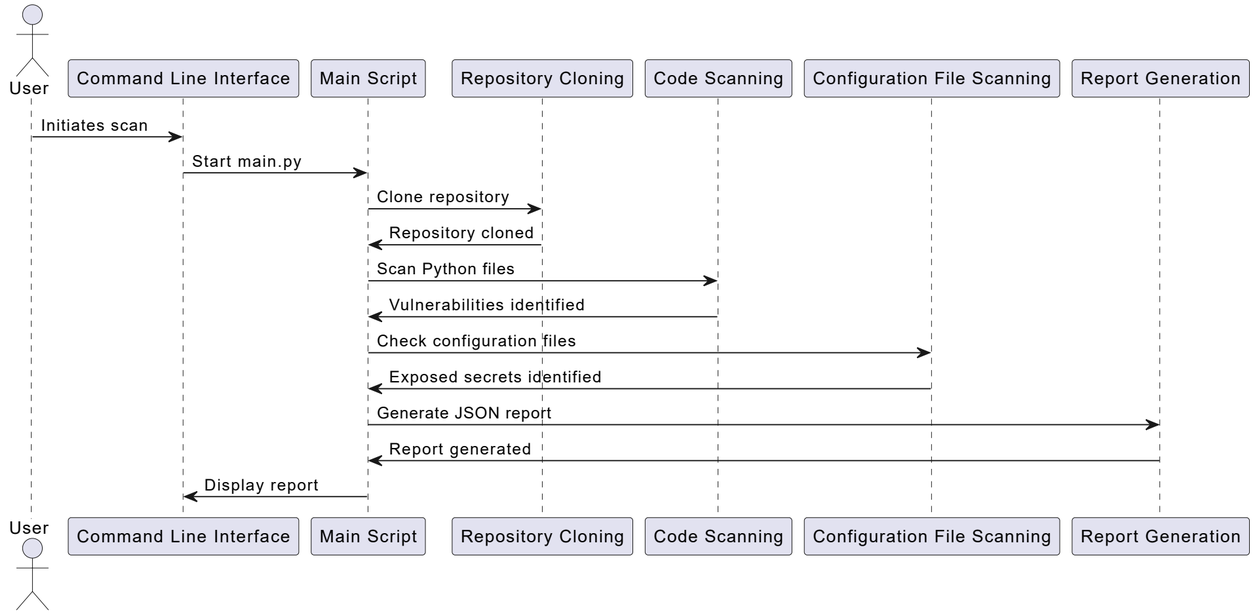
14) Workflow Diagram:

The workflow diagram illustrates the process of the Python security scanner. It starts with the input of either a GitHub URL or a local directory. If a URL is provided, the repository is cloned; otherwise, the local directory is used. The scanner then performs three main tasks: scanning Python files for vulnerabilities, checking configuration files for exposed secrets, and auditing dependencies. Finally, it compiles the results into a JSON report.



15) Communication Diagram:

The communication diagram illustrates the interaction between the user and various components of the Python security scanner. The user initiates the scan via the command line interface, which triggers the main script. The main script coordinates the repository cloning, code scanning, configuration file scanning, and report generation processes. Each component performs its task and returns the results to the main script, which ultimately generates a comprehensive security report for the user.


User
Command Line Interface
Main Script
Repository Cloning
Code Scanning
Configuration File Scanning
Report Generation
Initiates scan
Start main.py
Clone repository
Repository cloned
Scan Python files
Vulnerabilities identified
Check configuration files
Exposed secrets identified
Generate JSON report
Report generated
Display report
User
Command Line Interface
Main Script
Repository Cloning
Code Scanning
Configuration File Scanning
Report Generation